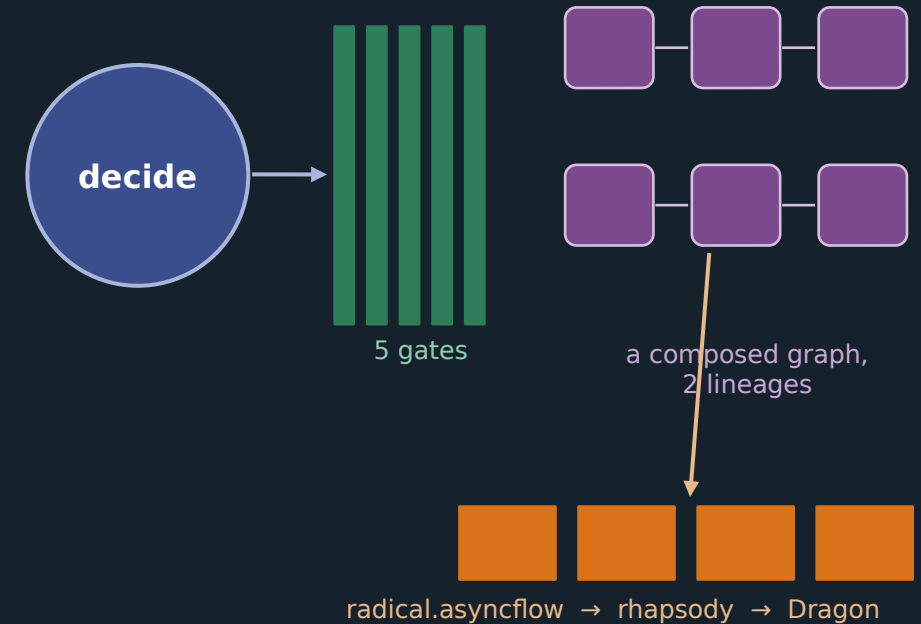


IMPRESS-A

An agent that composes its own workflows

— and what its first completed campaign on Delta did, and did not, prove

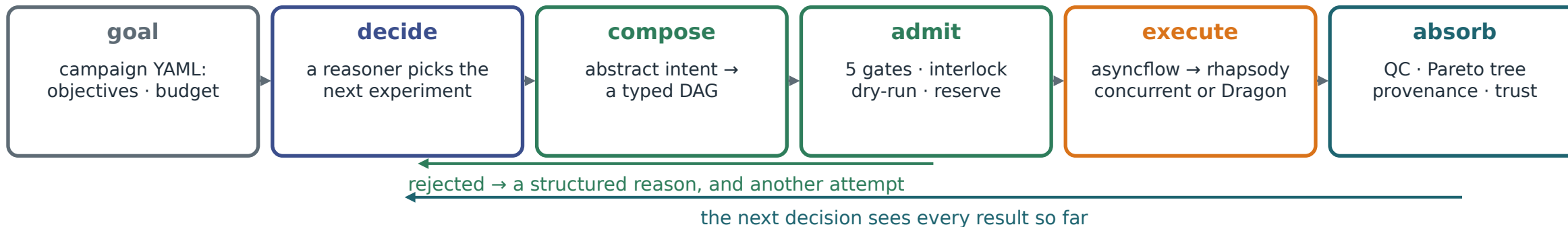
Job 22692304: six stages, 6/6 tasks ok in 2m53s, admitted on the first attempt, a one-node Pareto front carrying all four objectives



Lab code walk · main @ a810d67 · 6.5k lines of source, 152 tests, 11 tools in 5 toolkits

WHAT IT DOES

One goal, a campaign of composed experiments



What came back, for real — job 22692304

- rfd3_design → ligandmpnn_design → packmin → fastrelax → filter_shape → boltz_predict, 6/6 tasks ok in 2m53s
- all four campaign objectives valued: total_score -272, shape_complementarity 0.591, complex_plddt 0.507, ligand_ipitm 0.752
- admitted on the first attempt, 0.27 GPU-h and 0.06 CPU-h charged against the ledger
- per-tool cost measured for all six, and the specs corrected from it

And what one completed campaign is not

- One lineage, one cycle, one draw. replicas > 1 has never executed, so the independence invariant is unexercised.
- The node is suspect, not pass — the composition pattern is provisional. Nothing has ever been promoted on Delta, so the trusted path has never run.
- No measurement has ever superseded a prediction, so the calibration machinery is untested against reality.

Read it as a floor, not a result. The engineering claim is that a machine-composed workflow survived contact with real tools on real hardware and was caught, costed and recorded on the way through. The science is one design.

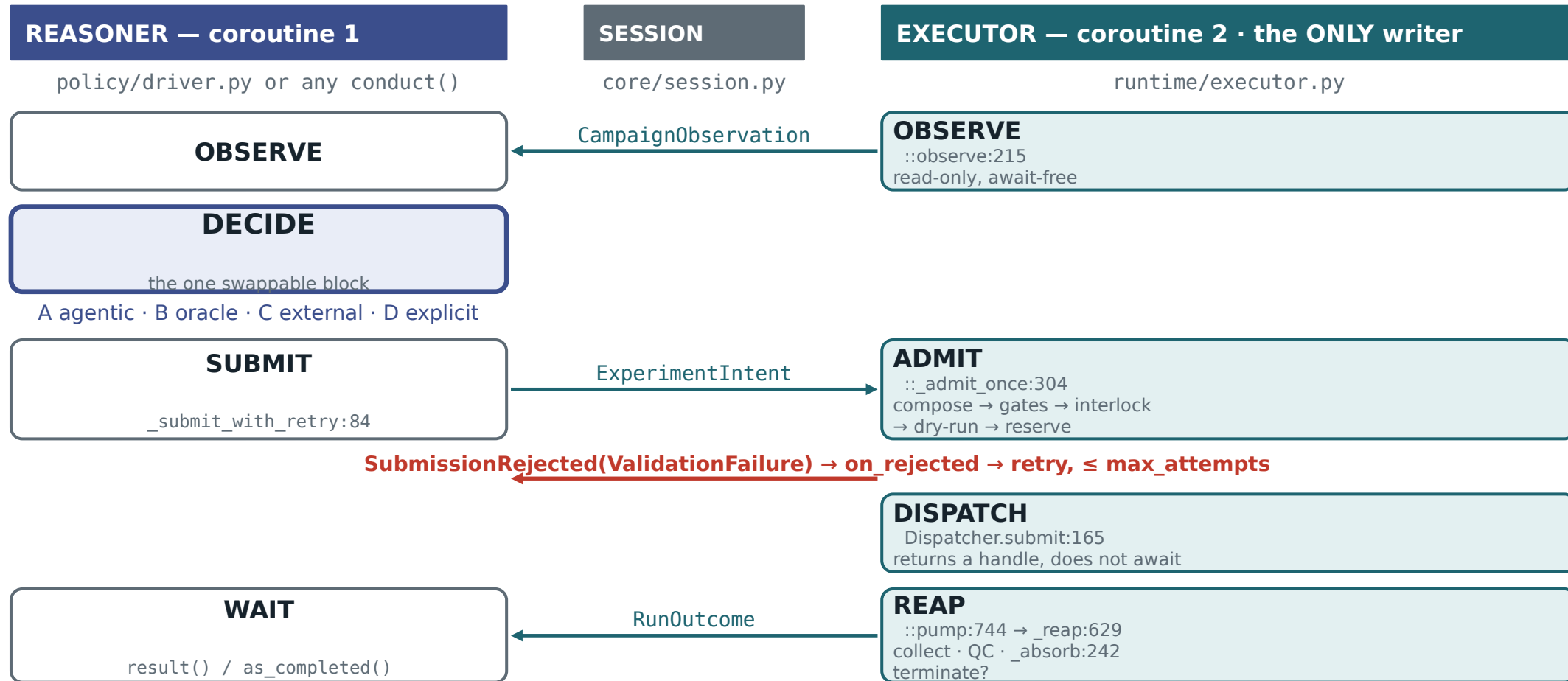
Two facts set the whole design

A machine composes workflows nobody has reviewed — out of scientific tools that fail **silently** far more often than they crash. A generator returns a structure with no secondary structure, a confident designability score, and exit code 0.

consequence	what it means in the code	where
A policy emits intent, never a DAG	ExperimentIntent is a list of tool ids and parameters. Turning it into a graph is the composer's job, so control models stay interchangeable — and the import contract forbids the edge.	policy/base.py · core/decision.py:21
Composition-time validation, then a dry-run	Five gates, then every task agent parameterizes without executing. Rejection is cheap and structured, so free composition is affordable to get wrong.	compose/validate.py:30, :143
Trust is scored on shape, and earned	A novel graph shape runs under a cost cap, a forced dry-run, and is marked suspect whatever it scores. Promotion is N consecutive clean runs; demotion is immediate.	compose/interlock.py:146
Exit code is never evidence	Every non-P6 tool declares mandatory QC gates, and a node whose verdict is FAIL is never eligible for the Pareto front, whatever it scored.	tools/agent.py:110 · core/pareto.py:56
One writer, and it decides termination	The executor owns all campaign state; every mutating block is await-free, so an observation is never taken mid-absorb. Budget and stagnation are facts a reasoner cannot see.	runtime/executor.py:242

The flip side, said plainly: if a human reviewed every graph before it ran, you would not build most of this. You would write the DAG down once and schedule it.

Two coroutines, one writer



Why the split exists

- The old manager awaited the whole graph where it submitted it, so the policy could not be consulted until the slowest lineage finished.

What keeps it honest

- Every mutating block in the executor is await-free. Adding an await inside observe or _absorb reintroduces torn reads — a discipline, not something Python enforces.

Layers, and the four arrows that are not there

control/	CampaignControlPlane · in-process ✓ HTTP+SSE ✓ MCP -- not built	
policy/	A agentic · B oracle · C external · D explicit · wrappers · driver	policy × tools policy × exec
manager.py	a facade: starts the executor and the reasoner, first to finish ends it	policy × runtime
runtime/	executor.py — sole state owner · session.py · runservice.py	
compose/	composer · graph · validate (5 gates) · interlock (trust)	compose × exec
tools/	spec · registry · agent (4 phases) · gates · the real adapters	
exec/	backend · dispatch · ledger · resources -- no caller	
core/	tree · pareto · qc · budget · decision · session · results · provenance	
	radical.asyncflow.WorkflowEngine → rhapsody backend	
	concurrent · dragon · radical · dask — chosen BY NAME from the campaign spec	

The two rules that carry weight

- policy → tools | exec | runtime — a policy that can reach a tool adapter stops emitting abstract intent, and control models stop being swappable.
- compose → exec — validation stays testable with no backend at all. That is the entire laptop test tier.

Asserted, not reviewed

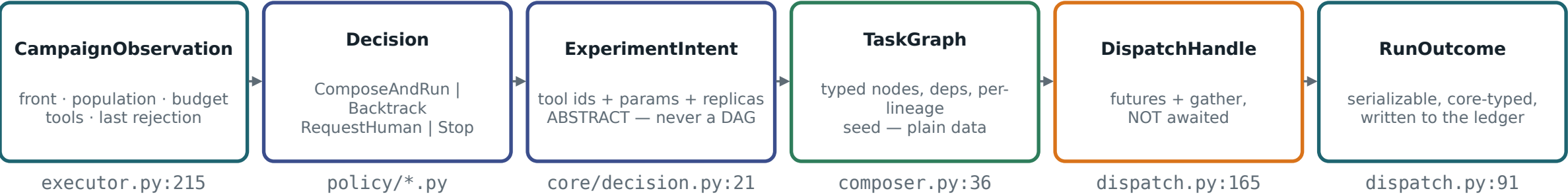
- tests/test_layering.py walks the source with ast and checks every internal import against one table.
- ast.walk deliberately: adapters defer science imports into run() bodies, and a deferred cross-layer import is exactly as fatal as one at the top.

What writing it found

- Two undocumented edges — manager → tools, and cli / __main__ in no row at all. Both are recorded now: a contract with undocumented edges is not a contract.

policy → core is the edge that forced core/session.py and core/results.py to exist: a reasoner may hold a RunOutcome, so RunOutcome cannot live in exec/.

One experiment, and the type at every hop



```
$ impress-a run campaigns/mock-stabilize.yaml --model D

campaign : mock-stabilize-D
cycles   : 3
stop     : target front size 3 reached
nodes    : 6  front: 4

run.json — mined from a real run, this checkout
```

What the first row is

- provisional pattern capped at 10% of available cpu_hours (1.500 > 1.200)
- The policy answered by shrinking from 3 lineages to 2 and resubmitting. That is on_rejected doing its job: a refusal costs an attempt, not the experiment.

run	shape	nodes	est. gpu-h	admitted?
r0001	9fdcf20...	12	0.165	refused — interlock
r0002	66bb4923...	8	0.110	yes
r0003	66bb4923...	8	0.110	yes
r0004	66bb4923...	8	0.110	yes

And what the rest of the run shows

- 6 design nodes over 3 cycles, every one marked suspect — the pattern was provisional until the third clean run promoted it (1 promotion, in trust_events).
- Front of 4; the campaign stopped on "target front size 3 reached" rather than on budget.

Admission: five gates, an interlock, and a reservation

gate	rejects	line
1 type	an edge whose producer type does not unify with the consumer's	:38
2 structure	cycles, empty graphs, a tool composed without its QC gates	:70
3 parameter	out of range, unknown, or declared frozen	:86
4 resource	a SCHEDULED P6, oversize shapes, unproven GPU API, external submission	:103
5 budget	an estimate exceeding remaining budget in any dimension	:134

then	what it does	where
INTERLOCK	one instance in flight for an untrusted shape; a cost cap for a provisional one — both transient	executor.py:331, :337
DRY-RUN	every task agent parameterizes WITHOUT executing	validate.py:143
RESERVE	re-check against AVAILABLE budget — remaining less what in-flight runs hold	executor.py:355

Why RESERVE is last: the dry-run awaits. By the time it returns, another submission may have claimed the budget gate 5 saw, so reserve re-checks and is the authoritative decision.

Be honest about the limit: the interlock buys examination and delay, not soundness. A *consistent* novel silent failure promotes.

A pattern is the graph's SHAPE

- tool ids plus typed edges, hashed. Parameter values are excluded, so tuning a parameter does not reset accumulated trust.

	trusted	provisional
cost ceiling	normal	10% of available
dry-run	skipped	forced
result	its QC verdict	SUSPECT anyway
in flight	many	one

Promotion is mechanical

- N consecutive clean runs promote; any failure demotes immediately and resets the counter.
- An untrusted shape may have only ONE instance in flight — N concurrent copies are one draw sampled N times, not N pieces of evidence.

★ One generic factory, and a submit that does not await

```
def _make_task(self, node_id, tool_id, params, invocation="", seed=None):
    """ONE generic factory for every node in every graph."""
    reg, spec = self.reg, self.reg.get(tool_id)
    agent_cls = reg.agent_for(tool_id)
    workdir = self.workdir      # bind locally: the closure is PICKLED

    async def _run(*deps):
        agent = agent_cls(spec)
        req = TaskRequest(tool=tool_id, params=params, seed=seed, ...)
        res = await agent(req)
        return {"tool": ..., "metrics": ..., "qc": ..., "cost": ...}

    _run.__name__ = node_id      # asyncflow reads the name from __name__
    return self.flow.function_task(_run)
```

exec/dispatch.py:144–163 · EDITED – annotations elided

```
for tid in g.topo_order():
    # edges via unawaited futures
    deps = [futures[d] for d in g.nodes[tid].deps]
    futures[tid] = tasks[tid](*deps, workflow_id=run_id)

# one bad task must not cancel its siblings
gather = asyncio.gather(*futures.values(), return_exceptions=True)
return DispatchHandle(run_id=run_id, graph_id=g.id,
                      futures=futures, gather=gather) # NOT awaited
```

exec/dispatch.py:182–190 · EDITED – see CODE_FOR_DECK.md

Two asyncflow idioms, not interchangeable

- An unawaited future passed as an argument expresses a DAG edge.
- `asyncio.gather` groups INDEPENDENT work — the replica lineages. The executor, not the dispatcher, decides when to wait on it, racing every in-flight gather with `FIRST_COMPLETED`.

The bet, and it held twice

- A DAG described as data dispatches with one generic factory — no codegen, no AST work, no templating.
- And the same factory survived going non-blocking unchanged: submit builds the gather and hands it back instead of awaiting it.

Three findings in four lines

- The task decorators take no `name=` kwarg — setting `__name__` first is what lets one factory serve every node.
- `workflow_id` tags every task, but no lookup API exists, so the handle is the run's own index.

Cancellation is advisory

- Measured, not assumed: `Future.cancel()` returns `False` once a callable has started, and `asyncflow` discards that answer. Queued work is reclaimed, running work is not, and you cannot learn which happened.
- So a run's terminal state always comes from collecting it. Backtracking sidesteps this by branching the tree; reclaiming a GPU from an abandoned run is unsolved.

★ Building the engine on the loop that will use it

```

async def make_engine_bounded(kind, config, timeout_s, heartbeat_s):
    if timeout_s <= 0 or kind.lower() in _CHEAP_KINDS:
        return await make_engine(kind, config)

    # ONLY the synchronous construction leaves this loop.
    threading.Thread(target=_construct_backend_in_thread,
                    args=(kind, config, fut), daemon=True).start()
    be = await asyncio.wait_for(asyncio.shield(async_fut), timeout_s)

    async def _finish():
        # on OUR loop, deliberately
        backend = await _init_backend(be) # __await__ registers states
        return await WorkflowEngine.create(backend=backend), backend

```

exec/backend.py:128–176 · EDITED — condensed, see CODE_FOR_DECK.md

Backends are awaitable, and must be awaited `__await__` triggers `_async_init()`, which registers task states. Constructing synchronously appears to work, then fails later with "Backend 'x' not registered. Available backends: []".

The engine must be built on the loop that will submit to it `WorkflowEngine.__init__` captures the running loop and puts its run-component dispatch task on it. Build one on a throwaway loop and you get an engine that looks fine and dispatches nothing — silently.

Real backends live in rhapsody, not asyncflow `asyncflow` exports only `Noop` and `Local`. Everything real resolves by NAME through `rhapsody.backends.get_backend()`, from the campaign spec — so no backend class is named outside one module.

Dragon empties the root logger `Pool()/ProcessGroup()` call `setup_BE_logging`, which clears root handlers. A run configured with `logging.basicConfig` alone goes mute the moment the backend comes up.

What each lesson cost

22328172
22328262

two full
allocations

the engine was built on a throwaway loop via `asyncio.run` on a helper thread — every submitted task hung

22318678

one 2-hour
allocation,
in full

`Dragon's Batch()` builds synchronously with no `await` points. No heartbeat, no campaign log, nothing, until the wall clock killed it

22491438

~64 GPU-hours,
37% of the bill

`flow.shutdown()` never returned after the science finished. Teardown is the more expensive end

Both ends are bounded now

- `backend_startup_timeout_s` / `_heartbeat_s` and `backend_shutdown_timeout_s`. A stuck teardown is abandoned with a warning, never raised — results are already durable by then.

A tool is a spec file, and loading it is validating it

```
id: filter_shape
pattern: P2 # compute pattern – normative, see docs
entry: impress_a.tools.rosetta_agents.FilterShapeAgent
resources: {cores: 2, walltime_s: 120}
inputs: {structure: {type: Complex}}
outputs: {structure: {type: Complex}}
metrics: [shape_complementarity]
qc_gates:
  - {id: output_present, params: {key: result}}
  - {id: metric_in_range, params: {metric: shape_complementarity,
                                   min: 0.55, max: 1.0}}
cost_model: {unit: invocation, cost: {cpu_hours: 0.01}}
```

toolkits/rosetta/tools/filter_shape/spec.yaml · EDITED



post_process ENFORCES the spec's qc_gates — never skippable, and execute() is not overridable by an adapter: pattern dispatch belongs to the exec layer.

Loading is validating

- A default outside its own range, an unknown QC gate id, a P1 tool declaring no GPU, a SKILL.md missing a required section — all are load-time errors.
- A toolkit that fails anywhere registers NOTHING, never a partial set.

The suite manufactures the failure mode

```
class NoodleAgent(TaskAgent):
    """A tool that LIES.

    Completes successfully, returns confident-looking numbers,
    and produces a structure with no secondary structure.
    """
    async def run(self, req, params):
        return {"result": "backbone", ...,
                "outputs": {"designs": "noodle"},
                "metrics": {"ss_fraction": 0.02, "designability": 0.91}}
```

tools/mock_agents.py:73–84 · EDITED

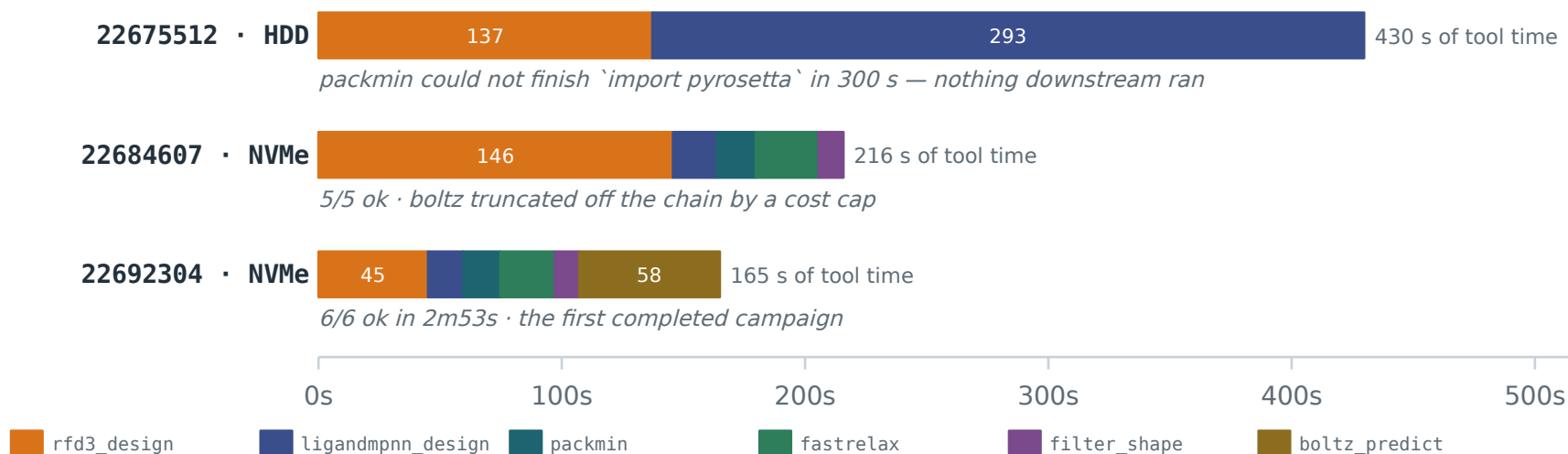
```
assert not res.failures, "the tool must SUCCEED - that is
                           what makes it silent"
assert res.merged_metrics()["designability"] > 0.9
assert res.merged_qc().verdict is QCVerdict.FAIL
```

test_campaign.py:81–83 · EDITED (one string wrapped)

Where this defence is still thin

- mock_noodle exists so the QC layer is tested at campaign scale against a tool that lies. The real toolkits have no equivalent.
- Their gates are almost entirely metric_in_range against a number the tool chose to report about itself — which is exactly what a confidently-wrong tool passes.
- No structural gates: nothing checks the ligand is in the output complex, that there are no chain breaks, or that sequence length matches the contig.
- RFD3 already reports ligand_clashes in a sidecar JSON we now open and do not read. That is the cheapest real structural gate available, and it is backlog B1/G2.

Four jobs to one completed campaign



Where the bytes live

- import pyrosetta off HDD-backed Lustre: 471 s, a 598 MB rosetta.so demand-paged. import torch: ~280 s.
- On NVMe both are seconds. That single variable is the whole difference between rows one and two.

Wall time is not stable

- rfd3_design took 145.7 s in 22684607 and 45 s in 22692304 — same campaign, same allocation shape, same parameters, 3.2× apart.

So do not tune cost models to it

- They feed gate 5 and the untrusted cap, which REFUSE graphs. They sit a few multiples above measurement on purpose.

Four defects that reached real hardware

1 What a tool writes

The *.cif.gz glob selected a diffusion TRAJECTORY, not the design — same extension, and `denoised` sorts first. A 5.7 MB multi-frame stack went downstream as the backbone, against a 19 KB design.

now: Discovery goes through the sidecar *_model_*.json now, and an out_dir it cannot identify a design in raises rather than guessing.

2 Whether a tool can import

LigandMPNN died in run.py's MODULE-LEVEL imports, before parsing an argument: a missing ml_collections, then np.int, removed in numpy 1.24 and still used by its vendored openfold.

now: It runs through MPNN_SHIM now — the shim restores the aliases and hands off via runpy — never run.py directly.

3 Where the bytes live

import pyrosetta cost 471 s off HDD, so packmin could not finish starting inside its 300 s budget. The queued fix was to raise three walltimes 6×, which would have hidden the real cost.

now: WORK_DIR on NVMe; `impress-a preflight` now flags a venv still on slow storage.

4 An estimate that refuses work

cost_model figures were literature guesses 6–69× over measurement. The inflated GPU side summed past the untrusted cap, the chain was refused, and the policy truncated boltz_predict off the end — silently.

now: Costs corrected from measurement, and _objectives_without_a_producer now WARNs. It is a heuristic, not a gate.

Two things to carry away. "Checked against the binary" is not "executed" — numbers 1 and 2 were both invisible to every contract check we had. And a plausible explanation is not a diagnosis: the trajectory bug was a convincing cause for a LigandMPNN failure it had nothing to do with.

The ledger that forgot — and what it still forgets

1 · Promotion was structurally unreachable on Delta, and it looked exactly like "not enough runs yet"

```
# executor.py loaded the ledger from Path(spec.root) / "_trust",
# and spec.root defaults to the RELATIVE "campaigns/_runs".
# delta_gpu_run.sh cds into $WORK_DIR/impress_a_runs/$SLURM_JOB_ID:

22684607/campaigns/_runs/_trust/cuda.jsonl    0 clean
22692304/campaigns/_runs/_trust/cuda.jsonl    1 clean
```

fixed in 52332c6 – CampaignSpec.trust_root

How it hid: nothing ever logged where the ledger was, so a file that silently reset looked identical to one that had not earned promotion yet — and it was reported that way. Campaign start now prints the absolute path with a pattern and trusted count. It had been true since the first Delta job, so **the trusted code path has never executed**: no pattern has ever skipped the forced dry-run, and the 10% cap has applied to every graph ever composed on real hardware.

2 · And a pattern's identity includes its BREADTH — found while building this deck

```
parts = sorted(
    # ONE string per NODE
    f"{n.tool}<-{'.'.join(sorted(self.nodes[d].tool for d in n.deps))}"
    for n in self.nodes.values()
)
return hashlib.sha256("".join(parts).encode()).hexdigest()[:16]
```

compose/graph.py:51 · VERBATIM + one added comment

N lineages give N duplicate parts, so the hash changes with the replica count — the way it deliberately does not change with a parameter:

promote_after = 3 counts within ONE file

- Two more jobs would have produced three files holding one run each, a pattern still untrusted, and two allocations spent for nothing.

the real six-stage chain	nodes	signature
replicas: 1	6	f5b21d82924fbcd0
replicas: 2	12	f8bcefed63c493ae
replicas: 4	24	78e730c67750a361
replicas: 1, num_designs: 3	6	f5b21d82924fbcd0 ← unchanged

What that costs, concretely

- Backlog A10's cheapest route — promote at replicas: 1, then run replicas: 4 — does not work. That is a different, untrusted pattern: still capped, refused at 0.68 vs 0.6 gpu-h, and on_rejected would silently shrink it to three lineages.

The question for the room

- Is trust earned at one lineage evidence about four, when the cap it would lift exists to bound blast radius?

Signatures computed live by slides/run_model.py against this checkout — the mock run on slide 6 shows the same shape, the shrink from 3 lineages to 2 producing a new pattern.

HOW TO RUN IT

Running it, and what the test tiers actually cover

```
source .venv/bin/activate          # NOT system python – see CLAUDE.md
pip install -e ".[dev]"

pytest tests -q                    # 152 tests, no allocation
impress-a tools                    # list the registered tools
impress-a preflight campaigns/<spec>.yaml # BEFORE sbatch
impress-a run campaigns/mock-stabilize.yaml --model D
# --model A|B|C|D|null|replay --no-guard drops the correction wrapper
```

file	what it covers
test_core.py	pure logic — tree, Pareto, budget, QC
test_validation.py	the five gates, the interlock, the trust ledger
test_campaign.py	full campaigns on a concurrent backend with mock tools
test_real_toolkits.py	the real specs, without executing any real binary
test_backend_bound.py	startup timeout/heartbeat against a fake backend that blocks its own thread's loop the way Dragon's Batch() does
test_http_adapter.py	a campaign driven end to end over a loopback socket
test_remote_session.py	a reasoner in another process, over the control plane
test_layering.py	the import contract, walked with ast rather than trusted to review
test_gate_fixtures.py	each real tool's gates against its known-bad fixtures, and the registry against the directories on disk

The whole local tier runs on a laptop

- A complete campaign — real manager, real policies, real composer and validator, stubbed science. HPC iteration is slow and expensive, so almost everything is verifiable locally.
- The corollary: everything only verifiable on HPC is unverified.

Two conventions worth stealing

- Counts are not restated in prose. test_gate_fixtures derives the tool count from the directories rather than asserting a number.
- A magic number in a test is often a bug report: stagnation_limit = 10_000 appeared in three tests before anyone noticed the defect was in the executor, not the setup.

A 20-second demo, if the room wants it

- `pytest tests -q -k lying` — the bundled tool that succeeds, reports 0.91 designability, and is failed by QC anyway.
- `impress-a run campaigns/delta-small-molecule-smoke.yaml --model D` must compose rfd3_design..., never mock_... — that confusion is the bug that made the smoke campaign necessary.

What is real, and what is not

Runs, for real	Has never run
A six-stage real-toolkit campaign ran end to end on Delta: 6/6 tasks, four objectives, a front.	replicas > 1 has NEVER executed. The independence invariant is the one most likely to be silently wrong, and it is unexercised.
The engine — compose, five gates, interlock, Pareto tree, provenance, durable run ledger.	Nothing has ever been promoted by the trust ledger, so the trusted path has never run.
Four control models, plus conduct() reasoners, over one shared executor.	No measurement has superseded a prediction — the calibration machinery is untested.
Control plane: in-process and HTTP+SSE, with synchronous admission, covered end to end.	No structural QC gate exists for any real tool; every real gate is a threshold on a self-reported number.
A reasoner in another process, driving a campaign over the HTTP plane.	No resume: the ledger and reattach() exist, nothing resumes from them.
Dragon startup AND teardown bounded; both measured against real lost allocations.	impress-a serve / reason do not exist, so --model C fails open and reports a model-D campaign as model C.
Per-tool cost measured for all six real tools, and the specs corrected from measurement.	sites/*.yaml is read by no code. MCP is designed, not built.

One campaign has completed. One lineage, one cycle, one draw — and the node it produced is marked suspect, which is the interlock working rather than a defect.

DISCUSSION

Six things I would like this room's opinion on

1 rhapsody's monitor loop drops a whole sweep

A worker exception that cannot be unpickled takes out every completion in that sweep. `batch_task.get()` is guarded; the follow-up `get_stdout(block=False)` re-reads the same entry and catches only `OSError`. Our half is fixed (`SubprocessError.__reduce__`). Should that handler catch `Exception` and fail the ONE task?

2 Why does Dragon's Batch() stall?

Bounded now, but not diagnosed. Job 22318678 produced only infra-connect lines for two hours. Best guess: GPU-affinity worker rendezvous, or OFI/libfabric negotiation under single-node mode. We now fail fast enough to iterate — what should we instrument?

3 Can a running task's GPU ever be reclaimed?

`Future.cancel()` returns `False` once started and `asyncflow` discards the answer, so we cannot even learn which happened. Is cooperative cancellation inside the task the only route, or is there a backend-level kill we should be using?

4 workflow_id tags tasks but nothing looks them up

So "which tasks belong to run R" is answered by our own handle, which cancellation and draining both need. Is a lookup API wanted upstream, or is the tag meant to stay a label?

5 Does breadth belong in a pattern's identity?

Slide 13. Parameters are deliberately excluded from the signature; replica count is not. Is trust earned at one lineage evidence about four, when the cap it would lift exists to bound blast radius?

6 What is the cheapest real structural gate?

Every real gate today is a threshold on a number the tool reports about itself. RFD3 gives us `ligand_clashes` and per-residue clash counts in a sidecar JSON we already open. For the Rosetta stages, upstream gates on `interaction_energy`, which we do not compute. Where would you start?

Everything above is reproducible from this checkout, and every file:line on these slides is greppable from your laptop while I talk.

Backup — the control plane and the remote reasoner

CampaignControlPlane — one protocol, adapters translate transport only

observe · events · steer · pause · resume · stop · artifacts · provenance · ingest_measurement · status · inflight · backtrack · request_human · submit_run · list_runs · run_result · await_run_result · cancel_run

in-process

control/plane.py::InProcessControlPlane

HTTP + SSE

control/http.py::ControlPlaneServer

MCP

the same protocol, a third transport

observe() returns what a policy sees

- The same CampaignObservation object a policy's decide() receives — informed monitoring means parity of evidence, not a progress bar.
- Admission is synchronous even over HTTP: 202 with a run id, or 409 with the gate and the reason that refused it.

Why admission cannot be deferred

- Accept-then-reject-later would sever a rejection from the request that caused it.
- And it cannot work anyway: the dry-run instantiates task agents, so admission has to happen where the toolkit is installed.

What is still missing — backlog C5, C6, C7

- RemoteSession and the routes it needs are covered end to end, but only tests/ ever constructs a server. There is no impress-a serve and no impress-a reason, so the split is reachable from the suite and not from a shell.
- Because of that, --model C builds an ExternalPolicy whose inbox can never be filled: it times out every 5 s and silently runs the model-D policy instead, while reporting itself as model C. It should refuse.
- And where the plane should live relative to the allocation is undecided — http.py is loopback, single trusted client, no auth, no TLS.

Backup — four control models over one executor

	model	mode	how decide() works	file
A	Four-node agentic loop	autonomous	A LangGraph graph: hypothesize → parameterize, auditable node by node	policy/agentic.py
B	Heavyweight LLM oracle	autonomous	One structured call over rich campaign state	policy/oracle.py
C	External caller steering	headless	Blocks on the control plane for a directive	policy/external.py
D	Explicit policy	autonomous	Rules, bandit, Bayesian optimization, or replay — fully reproducible	policy/explicit.py

They compose rather than inherit

- RuleCorrectionsPolicy wraps any policy in an unconditional external guard; LoggingPolicy wraps any policy for provenance. Decorators, not subclasses.
- Writing a fifth is subclassing BasePolicy and implementing one method.

The claim the whole design rests on

- Everything below the policy layer is byte-identical across A, B, C and D. A campaign run by a hand-written rule cascade exercises the same machinery as one steered by a frontier LLM.

One editorial point, since Flowgentic's authors are here

- The outer loop is still ours, deliberately — it is just two coroutines now instead of one. Ceding it to a framework pushes the validation gates, the dry-run and the interlock into awkward places.
- Flowgentic's own README reaches the same conclusion: it recommends the augmented rung — the application owns the loop, agents are bound at decision points — over the fully agentic one.